

DR BR Ambedkar National Institute of Technology



ARTIFICIAL INTELLIGENCE LAB (IT-515)

Submitted to:

Dr. Simranjeet Singh

Assistant professor

Submitted by:

Granth Gupta

M.Tech-D.A

26201812

INDEX

Sr. No.	Title	Date	Page No.	Signature
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				
17				
18				

1. Develop a simple rule-based expert system for animal identification. Define a set of facts and IF–THEN rules and infer the type of animal based on the given facts.

PYTHON CODE

```
# RULES

rules = [

    (["has_hair"], "mammal"),

    (["has_feathers"], "bird"),

    (["gives_milk"], "mammal"),

    (["lays_eggs", "has_feathers"], "bird")

]


# Check IF all conditions in a rule match the given facts

def checkTarget(target, rules):

    for conditions, fact in rules:

        if target == fact:

            return conditions

    return []


# ANIMAL IDENTIFICATION FUNCTION

def identifyAnimal(given_facts, rules) -> str:

    for conditions, animal_type in rules:

        match = True

        for attribute in conditions:
```

```

        if attribute not in given_facts:

            match = False

            break

    if match:

        return animal_type

    return "Current properties are insufficient to identify the animal"

# TESTING WITH USER INPUT

user_input = input("Enter facts separated by spaces (e.g., has_hair eats_meat):")
given_facts = user_input.strip().split()

result = identifyAnimal(given_facts, rules)

print("Type of animal: ", result)

```

OUTPUT

```

Enter facts separated by spaces (e.g., has_hair eats_meat): has_feathers

Type of animal: bird

```

2. Develop an interactive expert system for student course recommendation. The system should ask the user questions about marks, interests, and skills and recommend a suitable course using predefined IF–THEN rules.

PYTHON CODE

```

# RULES

RULES = [

    ({"interest": "technology", "skill": "logic", "min_marks": 75}, "BSc in CSE"),

    ({"interest": "healthcare", "skill": "research", "min_marks": 70}, "BSc in

```

```
Biotechnology"),
({"interest": "business", "skill": "logic", "min_marks": 60}, "BBA in
Finance"),
({"interest": "business", "skill": "communication", "min_marks": 60}, "BBA in
Marketing")
]
```

```
# RECOMMENDATION FUNCTION
```

```
def recommendCourse(facts: dict) -> str:
    for conditions, course in RULES:
        interest_match = facts["interest"] == conditions["interest"]
        skill_match = facts["skill"] == conditions["skill"]
        marks_match = facts["marks"] >= conditions["min_marks"]

        if interest_match and skill_match and marks_match:
            return "Recommended course is " + course

    return "Sorry! I don't have a recommended course for you."
```

```
# TESTING WITH USER INPUT
```

```
marks = float(input("Enter your marks: "))
interest = input("Enter your interest (technology, healthcare, business): ")
skill = input("Enter your skill (logic, research, communication): ")
```

```
user_facts = {
    "marks": marks,
    "interest": interest,
    "skill": skill
}
```

```
}

result = recommendCourse(user_facts)

print(result)
```

OUTPUT

```
Enter your marks: 80

Enter your interest (technology, healthcare, business): technology

Enter your skill (logic, research, communication): logic

Recommended course is BSc in CSE
```

3. Implement a forward-chaining inference engine for an expert system. Given a set of initial facts and IF–THEN rules, repeatedly apply applicable rules to derive new facts until the required conclusion is reached or no further rules can be applied.

PYTHON CODE

```
# KNOWN FACTS

known_facts_1 = ["has_fever", "has_cough"]

known_facts_2 = ["has_cough"]


# RULES

rules = [

    (["has_fever", "has_cough"], "has_flu"),

    (["has_flu"], "needs_rest")

]
```

```

def checkFacts(conditions, facts):
    for cond in conditions:
        if cond not in facts:
            return False
    return True

# FORWARD CHAIN

def forward_chaining(facts, rules, goal):
    facts_changed = True
    while facts_changed:
        facts_changed = False
        for conditions, result in rules:
            if checkFacts(conditions, facts) and result not in facts:
                facts.append(result)
                facts_changed = True
                if result == goal:
                    print(f"Goal '{goal}' successfully reached!")
                    return True

    if goal in facts:
        return True

    else:
        print(f"Could not reach the goal '{goal}'.")
        return False

# TESTING

target1 = "needs_rest"

print("Test 1")

```

```
print("Initial Facts: ", known_facts_1)

print("Target: ", target1)

forward_chaining(known_facts_1, rules, target1)

print("-----")
```

```
target2 = "has_flu"

print("Test 2")

print("Initial Facts: ", known_facts_2)

print("Targett: ", target2)

forward_chaining(known_facts_2, rules, target2)
```

OUTPUT

```
Test 1

Initial Facts:  ['has_fever', 'has_cough']

Target:  needs_rest

Goal 'needs_rest' successfully reached!

-----

Test 2

Initial Facts:  ['has_cough']

Target:  has_flu

Could not reach the goal 'has_flu'.
```

4. Develop an expert system for fault diagnosis using backward chaining. Given a suspected fault as the goal, identify the rules and facts required to prove or reject the hypothesis. The system should also display an explanation showing how the conclusion was reached.

PYTHON CODE


```

def checkTarget(target, rules):
    for conditions, fact in rules:
        if target == fact:
            return conditions
    return []

def backward_chaining(facts, rules, goal):
    if goal in facts:
        print(f"Fact '{goal}' is directly known.")
        return True

    conditions = checkTarget(goal, rules)

    if not conditions:
        print(f"No rule or fact found to prove '{goal}'.")
        return False

    print(f"To prove '{goal}', checking required conditions: {conditions}")

    for cond in conditions:
        proven= backward_chaining(facts, rules, cond)

        if not proven:
            print(f"Failed to prove condition '{cond}'. Hypothesis '{goal}'
                rejected.")
            return False

    print(f"All conditions met! Goal '{goal}' is PROVEN.")
    return True

facts = [
    "Lights are dim",
    "Engine does not turn over"
]

```

```

rules = [
    ("Starter clicks", "Lights are dim"], "Dead Battery"),
    ("Engine turns over", "Fuel tank is empty"], "Out of Gas"),
    ("Engine does not turn over"], "Starter clicks")
]

target = "Dead Battery"

is_proven = backward_chaining(facts, rules, target)

print("=====")

print(f"Hypothesis '{target}': {'CONFIRMED' if is_proven else 'REJECTED'}\n")

```

OUTPUT

To prove 'Dead Battery', checking required conditions: ['Starter clicks', 'Lights are dim']

To prove 'Starter clicks', checking required conditions: ['Engine does not turn over']

Fact 'Engine does not turn over' is directly known.

All conditions met! Goal 'Starter clicks' is PROVEN.

Fact 'Lights are dim' is directly known.

All conditions met! Goal 'Dead Battery' is PROVEN.

=====

Hypothesis 'Dead Battery': CONFIRMED